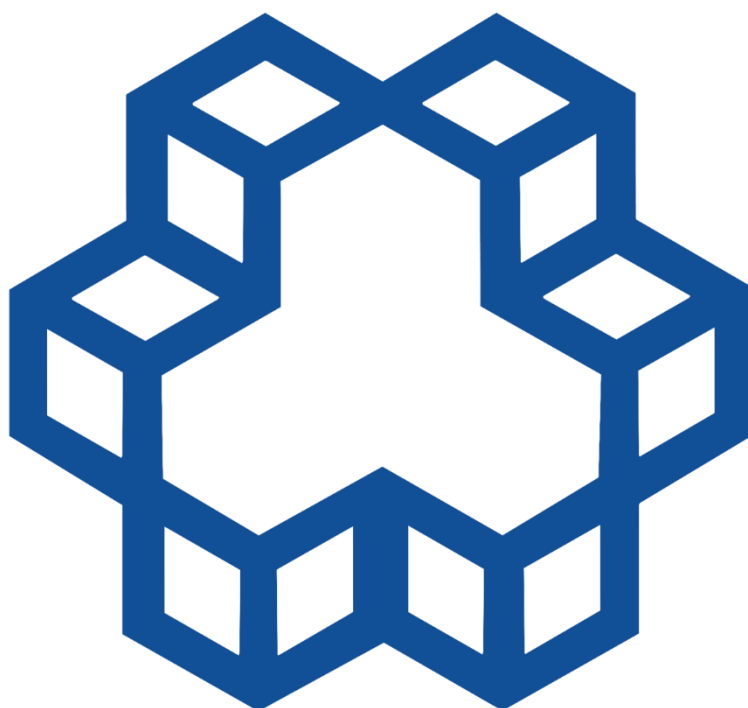


گزارش پروژه نهایی درس پایگاه داده‌ها

دانشگاه صنعتی خواجه نصیرالدین طوسی
دانشکده مهندسی کامپیوتر



عنوان پروژه:

پلتفرم جامع رزرو و خرید بلیط مسابقات ورزشی

استاد درس:

سرکار خانم دکتر پیشگو

نیمسال دوم سال تحصیلی ۱۴۰۴-۰۵

مخزن گیت‌هاب:

<https://github.com/koamz/sports-ticket-reservation.git>

اعضای تیم پروژه:

- سارینا ناصر مقدسی (شماره دانشجویی: 40222083)
- کوثر آموزگار (شماره دانشجویی: 40215133)
- علی کاشی‌پزها (شماره دانشجویی: 40224641)

۱. مقدمه و اهداف پروژه

پروژه پیش رو با هدف طراحی و پیاده‌سازی یک سامانه جامع، مقیاس‌پذیر و بدون واسطه برای رزرو و خرید بلیط مسابقات ورزشی (فوتبال، والیبال، بسکتبال و...) توسعه داده می‌شود. اهداف کلیدی این سامانه عبارتند از:

- مدیریت صندلی‌ها و ظرفیت‌های سالن‌ها و ورزشگاه‌ها به تفکیک ویژگی‌های اختصاصی هر ورزش.
- پیاده‌سازی مکانیزم رزرو موقت بلیط (Ticket Lock) با استفاده از بستر ذخیره‌سازی حافظه‌محور Redis جهت جلوگیری از تعارض همزمانی (Race Condition).
- ارائه یک ساختار کاملاً نرمال‌سازی شده در لایه پایگاه داده رابطه‌ای (PostgreSQL) برای حفظ یکپارچگی تراکنش‌ها (ACID).
- تضمین کیفیت و پایداری کدهای دیتابیس از طریق پیاده‌سازی تست‌های یکپارچه‌سازی خودکار در خط لوله CI/CD.

۲. فاز اول: طراحی نمودار رابطه موجودیت‌ها (ERD)

در طراحی جدید نمودار رابطه موجودیت‌ها (ERD)، هدف اصلی دستیابی به ساختاری کاملاً نرمال‌سازی شده بر اساس فرم‌های استاندارد دیتابیس (تا سطح 3NF) و برطرف کردن هرگونه وابستگی انتقالی بوده است.

۲.۱. تغییرات کلیدی در راستای بهبود نرمال‌سازی (3NF)

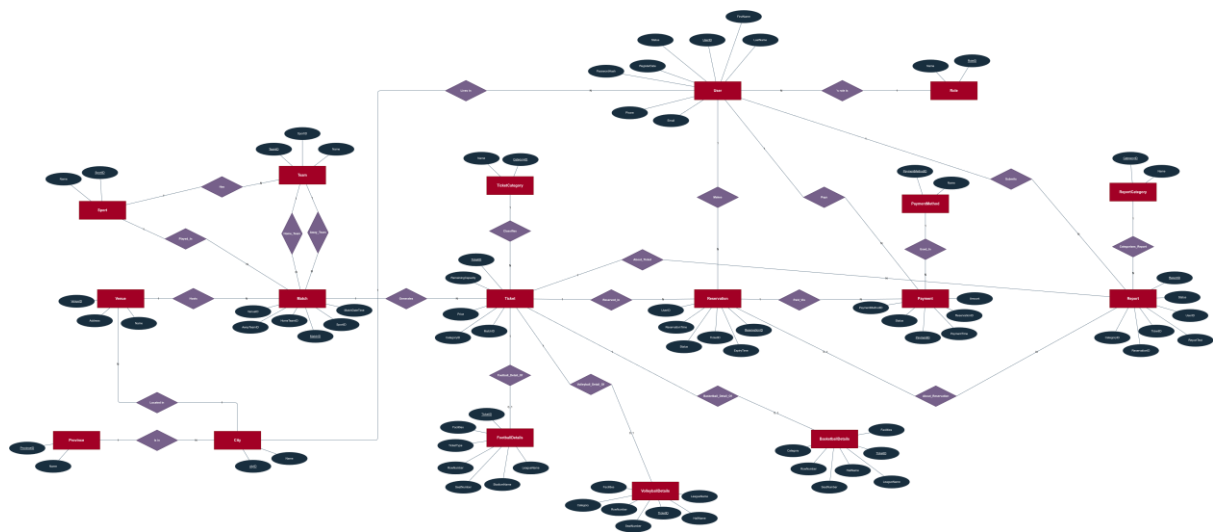
- **انتزاع نقش‌ها (ROLE):** نقش‌های کاربران از حالت متنی ساده خارج شده و به یک جدول مستقل تبدیل شدند تا دسترسی‌ها و سطوح امنیتی در آینده به صورت پویا قابل مدیریت باشند.
- **انتزاع ورزش‌ها (SPORT):** نوع ورزش‌ها به عنوان یک کاتالوگ مستقل تعریف شد تا از تکرار فیلدهای متنی نامنظم در مسابقات جلوگیری شود.
- **موجودیت مستقل تیم‌ها (TEAM):** تیم‌های ورزشی به عنوان یک موجودیت مستقل و در رابطه با جدول ورزش‌ها تعریف شدند تا اطلاعات آن‌ها دچار افزونگی نگردد.

- جدول سازی برای دسته بندی بلیطها (TICKET_CATEGORY): دسته بندی قیمت و جایگاه بلیطها (عادی، VIP، ویژه) با رابطه کلید خارجی ساختاریافته شد.

- بهبود ارث بری ویژگی های ورزشی (Class Table Inheritance): برای سه ورزش فوتبال، والیبال و بسکتبال، جداول جزئیات اختصاصی طراحی شدند که رابطه یک به یک با جدول پایه TICKET دارند. هر جدول زیرگروه، فیلدهای مکان محور (بخش، ردیف و صندلی) و امکانات ویژه خود را به شکل ایزوله مدیریت می کند.

۲.۲. نمودار رابطه موجودیتها

نمودار فیزیکی روابط پایگاه داده به شرح زیر طراحی گردیده است:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
○ sarina@sarina:~/Projects/sports-ticket-reservation$ docker exec -it sports_ticket_postgres psql -U test_user -d sports_ticket_db
psql (15.19)
Type "help" for help.

sports_ticket_db=# \dt
               List of relations
Schema |      Name      | Type | Owner
-----|-----|-----|-----
public | basketball_details | table | test_user
public | cities          | table | test_user
public | football_details | table | test_user
public | matches         | table | test_user
public | payments        | table | test_user
public | provinces       | table | test_user
public | reports         | table | test_user
public | reservations     | table | test_user
public | roles           | table | test_user
public | sports          | table | test_user
public | teams           | table | test_user
public | ticket_categories | table | test_user
public | tickets         | table | test_user
public | users           | table | test_user
public | venues          | table | test_user
public | volleyball_details | table | test_user
(16 rows)

sports_ticket_db=# SELECT * FROM provinces;
 id |  name_en  |  name_fa
-----|-----|-----
  1 | Tehran    | تهران
  2 | Isfahan   | اصفهان
  3 | Fars      | فارس
  4 | Khorasan Razavi | خراسان رضوی
  5 | East Azerbaijan | آذربایجان شرقی
  6 | Alborz    | البرز
  7 | Mazandaran | مازندران
  8 | Gilan     | گیلان
  9 | Khuzestan | خوزستان
 10 | Yazd     | یزد
 11 | Kerman   | کرمان
 12 | Hamedan  | همدان
(12 rows)

sports_ticket_db=#
```

نمایش لیست جداول پایگاه داده (\dt) در محیط ترمینال داکر و تست صحت داده‌های جدول استان‌ها (provinces).

۳. فاز دوم: طراحی جداول، قیدها و کوئری‌ها

۳.۱. قیدهای صحت و یکپارچگی داده‌ها (Constraints)

برای اطمینان از صحت داده‌ها در سطح موتور دیتابیس (پستگرس)، قیدهای زیر اعمال شده‌اند:

- **قید چک برای قیمت و ظرفیت بلیط:** قیمت هر بلیط باید بزرگتر یا مساوی صفر باشد. همچنین ظرفیت باقی‌مانده نباید منفی بوده یا از ظرفیت کل فراتر رود.
- **اعتبارسنجی ساختار ایمیل و تلفن:** در جدول users با استفاده از الگوهای Regular Expression، صحت ساختاری ایمیل و شماره‌های تماس ورودی کنترل می‌شود.
- **رزرو یکتای صندلی:** برای جلوگیری از رزرو همزمان یک صندلی توسط دو کاربر، فیلد ticket_id در جدول reservations به صورت یکتا (UNIQUE) تعریف شده است.
- **محدودیت عدم رویارویی تیمی یکسان:** در جدول matches با قید چک بررسی می‌شود که تیم میزبان و میهمان برابر نباشند.

۳.۲. بهینه‌سازی و ایندکس‌گذاری (Query Optimization)

به منظور تسریع فرآیند جستجوی بلیط‌ها بر اساس تاریخ مسابقه و شماره تماس کاربران، ایندکس‌های زیر تعریف شدند:

- **idx_matches_time** بر روی فیلد زمان برگزاری مسابقات.
- **idx_payments_date_new** بر روی فیلد زمان ثبت پرداخت‌ها.
- **idx_users_email** و **idx_users_phone** برای تسریع فرآیند ورود و احراز هویت.

۳.۳. پرس‌وجوهای اطلاعاتی و تحلیلی (DQL)

تعداد ۲۲ کوئری تحلیلی درخواستی پروژه با هدف استخراج گزارش‌های مدیریتی پیچیده، تحلیل عملکرد پشتیبان‌ها و رفتارهای خرید کاربران پیاده‌سازی شده‌اند. این کوئری‌ها در پوشه کدهای پروژه تحت مسیر `database/queries/analytical_queries.sql` مستقر گشته‌اند. برخی از مهم‌ترین تکنیک‌های استفاده‌شده در این بخش عبارتند از:

- **Common Table Expressions (CTE):** جهت محاسبه و تفکیک مجموع پرداخت‌های کاربران و مقایسه با میانگین کل (کوئری ۶ و ۱۷).
- **تقسیم رابطه‌ای (Relational Division):** جهت شناسایی وفادارترین کاربران سیستم که از تمامی انواع مسابقات ورزشی بلیط تهیه کرده‌اند (کوئری ۱۴).

```
sports ticket db=# SELECT u.first_name, u.last_name
sports ticket db=# FROM users u
sports ticket db=# JOIN roles r ON u.role_id = r.id
sports ticket db=# WHERE r.name = 'visitor' AND u.id NOT IN (SELECT DISTINCT user_id FROM reservations);
 first_name | last_name
-----
 Mohammad  | Ahmadi
 Sara       | Karimi
 Reza       | Mohammadi
 Zahra      | Hosaini
 Ehsan      | Nouri
 Amir      | Taji
 Neda       | Amini
 Hassan     | Shakeri
 Hadi       | Moradi
 Fatemeh    | Karimi
 Ali        | Rezaei
(12 rows)

sports ticket db=# SELECT DISTINCT u.first_name, u.last_name
sports ticket db=# FROM users u
sports ticket db=# JOIN payments p ON u.id = p.user_id
sports ticket db=# WHERE p.status = 'success';
 first_name | last_name
-----
 Ali        | Rezaei
(1 row)

sports ticket db=# SELECT u.id, u.first_name, u.last_name,
sports ticket db=#          TO_CHAR(p.paid_at, 'YYYY-MM') AS payment_month,
sports ticket db=#          SUM(p.amount) AS total_paid
sports ticket db=# FROM users u
sports ticket db=# JOIN payments p ON u.id = p.user_id
sports ticket db=# WHERE p.status = 'success'
sports ticket db=# GROUP BY u.id, u.first_name, u.last_name, TO_CHAR(p.paid_at, 'YYYY-MM')
sports ticket db=# ORDER BY u.id, payment_month;
 id | first_name | last_name | payment_month | total_paid
-----
  1 | Ali        | Rezaei   | 2025-05       | 1320000.00
(1 row)

sports ticket db=# []
```

خروجی اجرای پرس‌وجوهای پیشرفته و تحلیلی روی کاربران، پرداخت‌ها و تفکیک مجموع تراکنش‌های موفق ماهانه

```
sports ticket db=# SELECT u.id, u.first_name, u.last_name, c.name_fa AS city_name
sports ticket db=# FROM users u
sports ticket db=# JOIN payments p ON u.id = p.user_id
sports ticket db=# JOIN reservations r ON p.reservation_id = r.id
sports ticket db=# JOIN tickets t ON r.ticket_id = t.id
sports ticket db=# JOIN matches m ON t.match_id = m.id
sports ticket db=# JOIN venues v ON m.venue_id = v.id
sports ticket db=# JOIN cities c ON v.city_id = c.id
sports ticket db=# WHERE p.status = 'success'
sports ticket db=# GROUP BY u.id, u.first_name, u.last_name, c.id, c.name_fa
sports ticket db=# HAVING COUNT(r.id) = 1;
 id | first_name | last_name | city_name
-----+-----+-----+-----
  1 | Ali       | Rezaei   | امپهان
  1 | Ali       | Rezaei   | تبریز
(2 rows)

sports ticket db=# SELECT u.*
sports ticket db=# FROM users u
sports ticket db=# JOIN payments p ON u.id = p.user_id
sports ticket db=# WHERE p.status = 'success'
sports ticket db=# ORDER BY p.paid_at DESC
sports ticket db=# LIMIT 1;
 id | role_id | city_id | first_name | last_name | email | phone | password_hash | status | created_at
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----
  1 |      1 |      1 | Ali       | Rezaei   | ali@gmail.com | +98912111111 | $2b$10$tJ9F0/bE/W5FpXQ0F68N.03gX/VvHe8R6nZszV.fHeXg8lBvB.X0q | active | 2023-01-10 10:00:00
(1 row)

sports ticket db=# WITH user_totals AS (
sports ticket db=#   SELECT user_id, SUM(amount) AS total_amount
sports ticket db=#   FROM payments
sports ticket db=#   WHERE status = 'success'
sports ticket db=#   GROUP BY user_id
sports ticket db=# )
sports ticket db=# SELECT u.first_name, u.last_name, u.email, u.phone
sports ticket db=# FROM users u
sports ticket db=# JOIN user_totals ut ON u.id = ut.user_id
sports ticket db=# WHERE ut.Total_amount > (SELECT AVG(total_amount) FROM user_totals);
 first_name | last_name | email | phone
-----+-----+-----+-----
(8 rows)

sports ticket db=#
```

خروجی کوثری‌های پیشرفته شامل محدودسازی نتایج و محاسبات مقایسه‌ای

۳.۴. توابع و پروسجرها (PL/pgSQL Functions)

تعداد ۸ تابع ذخیره‌شده انعطاف‌پذیر برای تسهیل کارهای پرتکرار سمت سرور طراحی شده‌اند. این توابع با دریافت ورودی‌های پویا، خروجی‌های ساختاریافته در قالب جدول (RETURNS TABLE) ارائه می‌دهند. کدهای مربوطه در آدرس `database/procedures/stored_procedures.sql` قرار دارند.

۴. تضمین کیفیت و تست یکپارچه‌سازی پایگاه داده

برای اولین بار در طراحی این پایگاه داده، یک بستر تست خودکار یکپارچه‌سازی (Database Integration Testing) با استفاده از فریم‌ورک پایتونی **Pytest** و درایور رسمی **Psycopg2** پیاده‌سازی گردید. این بستر تست تضمین می‌کند که هرگونه تغییر در ساختار فیزیکی دیتابیس، قیود داده‌ای، کوثری‌های تحلیلی یا منطق پروسجرها، درجا مورد سنجش قرار گرفته و صحت عملکرد آن‌ها تایید می‌شود.

۴.۱. شرح تست‌های پیاده‌سازی شده (۳۳ سناریوی جامع)

تعداد ۳۳ تست یکپارچه‌سازی جامع در فایل `tests/db/test_database.py` پیاده‌سازی شده است که سناریوهای فیزیکی، منطقی و پروسجرهای ذخیره‌شده را به طور خودکار بر روی دیتابیس فعال بررسی می‌کنند:

ردیف	نام متد تست	دسته‌بندی تست	شرح عملکرد سناریوی تست
۱	test_negative_ticket_price_constraint	قید CHECK	بررسی رفتار دیتابیس در صورت تلاش برای درج بلیط با قیمت منفی
۲	test_invalid_email_format_constraint	قید CHECK	اعتبارسنجی ساختار Regex ایمیل برای ممانعت از ورود داده‌های هرز

ردیف	نام متد تست	دسته‌بندی تست	شرح عملکرد سناریوی تست
۳	test_unique_seat_reservation	قید UNIQUE	ممانعت از رزرو همزمان یک صندلی مشخص برای دو کاربر مجزا
۴	test_query_01_no_reservations	کوئری ۱ (DQL)	استخراج اطلاعات دقیق تماشاگرانی که تا کنون هیچ صندلی رزرو نکرده‌اند
۵	test_query_02_at_least_one_purchase	کوئری ۲ (DQL)	استخراج مشخصات کاربرانی با حداقل یک خرید موفق در پایانه پرداخت
۶	test_query_03_monthly_user_payments	کوئری ۳ (DQL)	دسته‌بندی و محاسبه مجموع پرداخت‌های ماهانه هر کاربر به تفکیک زمان
۷	test_query_04_single_purchase_per_city	کوئری ۴ (DQL)	شناسایی کاربرانی با الگوهای رفتاری یک خرید واحد در سطح هر شهر
۸	test_query_05_newest_ticket_buyer	کوئری ۵ (DQL)	استخراج اطلاعات هویتی کاربری که جدیدترین تراکنش موفق را ثبت کرده است
۹	test_query_06_payments_above_average	کوئری ۶ (DQL)	فیلترینگ کاربرانی با مجموع خرید بیشتر یا مساوی با میانگین کل پرداخت‌ها
۱۰	test_query_07_tickets_sold_per_sport	کوئری ۷ (DQL)	گزارش آماری بلیط‌های نهایی شده به تفکیک سه رشته ورزشی سیستم
۱۱	test_query_08_top_buyers_last_week	کوئری ۸ (DQL)	استخراج ۳ تماشاگر برتر با بالاترین نرخ خرید در بازه زمانی ۷ روز گذشته
۱۲	test_query_09_tickets_sold_in_tehran_cities	کوئری ۹ (DQL)	گزارش تفکیکی بلیط‌های فروخته شده در شهرستان‌های استان تهران
۱۳	test_query_10_oldest_user_purchase_cities	کوئری ۱۰ (DQL)	شناسایی مقصدهای خرید اولین و قدیمی‌ترین تماشاگر ثبت‌نام شده
۱۴	test_query_11_support_staff_list	کوئری ۱۱ (DQL)	استخراج و تفکیک مشخصات هویتی کارکنان لایه پشتیبانی سیستم

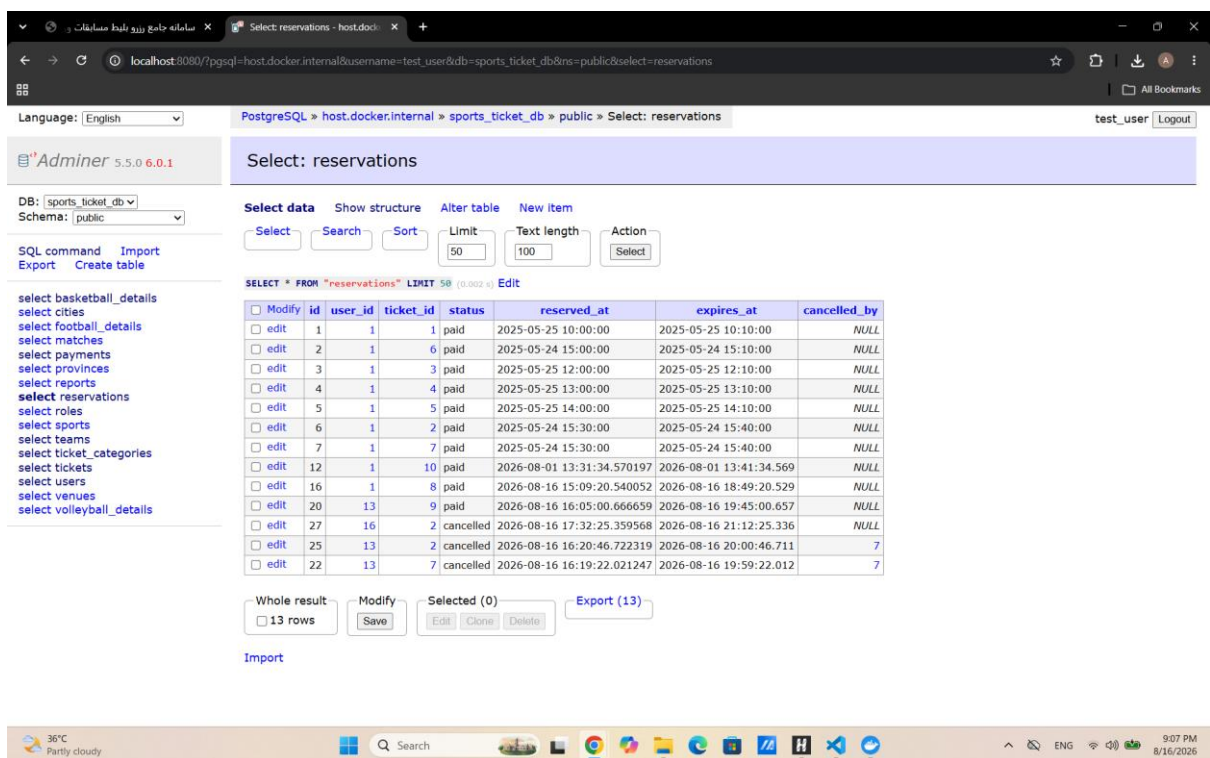
ردیف	نام متد تست	دسته‌بندی تست	شرح عملکرد سناریوی تست
۱۵	test_query_12_users_with_multiple_purchases	کوئری ۱۲ (DQL)	استخراج لیست تماشاگران وفادار با حداقل ۲ تراکنش موفق ثبت شده
۱۶	test_query_13_football_purchases_limit	کوئری ۱۳ (DQL)	کنترل تماشاگرانی با سقف خرید حداکثر ۲ صندلی در رشته ورزشی فوتبال
۱۷	test_query_14_all_sports_buyers	کوئری ۱۴ (DQL)	شناسایی خریداران همه جانبه که از تمامی ورزش های موجود بلیط تهیه کرده اند
۱۸	test_query_15_today_purchases_by_time	کوئری ۱۵ (DQL)	گزارش خریدهای جاری روز با ترتیب دقیق زمانی ساعت ثبت تراکنش
۱۹	test_query_16_second_best_selling_ticket	کوئری ۱۶ (DQL)	تحلیل و استخراج دومین مسابقه پرفروش جهت ارزیابی آماری تقاضا
۲۰	test_query_17_highest_support_cancellations	کوئری ۱۷ (DQL)	ارزیابی عملکرد و استخراج پشتیبانی با بیشترین لغو رزرو به همراه درصد کل لغوها
۲۱	test_query_18_update_user_last_name_to_reddington	کوئری ۱۸ (DML)	تغییر خودکار فامیلی پرلغاترین کاربر به ردینگتون) به همراه Rollback تراکنش)
۲۲	test_query_19_delete_cancelled_reservations_of_reddington	کوئری ۱۹ (DML)	حذف رزروهای منقضی شده کاربر ردینگتون از دیتابیس) به همراه Rollback)
۲۳	test_query_20_delete_all_cancelled_reservations	کوئری ۲۰ (DML)	پاک سازی کامل تمام رزروهای کنسل شده در کل سیستم) به همراه Rollback)
۲۴	test_query_21_discount_tickets_sold_yesterday_in_azadi	کوئری ۲۱ (DML)	اعمال تخفیف ۱۰ درصدی صندلی های فروخته شده روز قبل ورزشگاه آزادی (Rollback)
۲۵	test_query_22_most_reported_ticket_details	کوئری ۲۲ (DQL)	استخراج موضوعات و فراوانی گزارش های ثبت شده برای پرگزارش ترین بلیط
۲۶	test_sp_01_get_user_purchased_tickets	پروسجر ۱	تایید صحت اجرای تابع استخراج بلیط های خریدار منطبق بر ستون های چندزبانه

ردیف	نام متد تست	دسته بندی تست	شرح عملکرد سناریوی تست
۲۷	test_sp_02_get_cancelled_users_by_support	پروسجر ۲	سنجش صحت تابع لیست کردن لغوها به تفکیک ایمیل پشتیبان
۲۸	test_sp_03_get_tickets_purchased_in_city	پروسجر ۳	صحت سنجی تابع فیلترینگ و بازگشت تیکت ها بر اساس نام شهر
۲۹	test_sp_04_search_tickets	پروسجر ۴	تست عملکرد موتور جستجوی متنی دیتابیس روی ستون های انگلیسی و فارسی
۳۰	test_sp_05_get_fellow_citizens	پروسجر ۵	تایید عملکرد تابع هم شهری ها بدون نمایش رکورد خود متقاضی
۳۱	test_sp_06_get_top_buyers_since	پروسجر ۶	تست خروجی تابع خریداران برتر از تاریخ مبدا بر اساس رتبه بندی تعداد
۳۲	test_sp_07_get_cancelled_tickets_by_sport	پروسجر ۷	تست تابع فیلترینگ صندلی های کنسل شده به تفکیک رشته ورزشی
۳۳	test_sp_08_get_users_with_most_reports_by_subject	پروسجر ۸	تایید صحت خروجی کاربران پر گزارش به تفکیک موضوع موضوعی گزارش

۴.۲. ایزوله سازی تراکنش ها و همگام سازی چندزبانه

در طراحی و اجرای این لایه تست، دو استراتژی فنی مهم به کار رفته است:

- **مدیریت تراکنش ها (Rollback Strategy):** برای تمامی تست هایی که شامل تراکنش های باز نویسی دیتابیس، آپدیت یا حذف فیزیکی داده ها هستند (تست های ۲۱ الی ۲۴)، پس از اجرای موفقیت آمیز هر متد تست، بلافاصله دستور `db_connection.rollback()` فراخوانی می شود تا داده های تستی اولیه بدون تغییر و کاملاً تمیز برای بقیه تست ها باقی بمانند.



- همگام‌سازی با ستون‌های چندزبانه: تمامی کوئری‌های تحلیلی و پروسجرها با ستون‌های جدید دیتابیس همگام‌سازی شده‌اند و در زمان آزمون، فیلدهای فارسی (_fa) و انگلیسی (_en) را به طور دقیق در روابط پیوند (JOIN) دیتابیس کنترل می‌کنند.

۵. فاز سوم: پیاده‌سازی سمت سرور و معماری نرم‌افزار

در این فاز، بخش بک‌اند پروژه بر اساس معماری چندلایه (Layered Architecture) و بر پایه اصول طراحی تمیز (Clean Design) پیاده‌سازی گردید. با توجه به الزامات فنی دانشگاه، هیچ‌گونه ابزار ORM در این پروژه به کار نرفته و تمامی فرآیندهای تعامل با دیتابیس از طریق کوئری‌های خام SQL و به صورت پارامتری (Parameterized Queries) برای جلوگیری از آسیب‌پذیری تزریق کد (SQL Injection) انجام شده است.

۵.۱. ساختار لایه‌های نرم‌افزار (Separation of Concerns)

بخش بک‌اند به چهار لایه‌ی مجزا با وظایف کاملاً مستقل تقسیم شده است:

۱. لایه مسیرها و کنترلرها (Routes & Controllers): وظیفه دریافت درخواست‌های HTTP، اعتبارسنجی اولیه فرمت داده‌ها و ارسال پاسخ‌های استاندارد JSON به کلاینت را بر عهده دارد.
۲. لایه منطق تجاری (Services): هسته اصلی منطق برنامه است. محاسبات مالی، درصد‌های جریمه، مدیریت نشست‌ها، و یکپارچه‌سازی هماهنگ بین پایگاه داده دیسک (PostgreSQL) و حافظه موقت (Redis) در این لایه انجام می‌شود.
۳. لایه ریپازیتوری (Repositories): تنها لایه‌ای است که به صورت مستقیم با درایور پایگاه داده pg در ارتباط است و کوئری‌های خام SQL را اجرا می‌کند.

۴. لایه مدل‌ها (Domain Models): کلاس‌های دامنه و توابع اعتبارسنجی مستقل (مانند بررسی قوی پسوردها، صحت الگوهای ایمیل و تلفن) در این لایه به دور از وابستگی‌های دیتابیزی تعریف شده‌اند.

۶. منطق‌های تجاری، کشینگ پویا و یکپارچه‌سازی Redis

ذخیره‌ساز حافظه‌محور Redis در این پروژه دو نقش حیاتی بازی می‌کند: کش سرور (Cache Server) برای بهینه‌سازی سرعت و مدیریت وضعیت‌های موقت (State Management) برای احراز هویت و قفل بلیط‌ها.

۶.۱. احراز هویت دو مرحله‌ای و مدیریت OTP

فرآیند لاگین و ثبت‌نام کاملاً بر بستر رمزهای یک‌بار مصرف (OTP) طراحی شده است:

- به محض درخواست کاربر، یک رمز تصادفی ۶ رقمی تولید شده و در ردیس تحت کلید `otp:[contact]` ذخیره می‌گردد.
- این کلید دارای زمان اعتبار سخت‌گیرانه ۵ دقیقه (TTL: 300s) است و پس از آن به صورت خودکار توسط ردیس نابود می‌شود.
- پس از تایید موفقیت‌آمیز کد توسط کاربر، کلید مربوطه بلافاصله برای جلوگیری از استفاده مجدد (Replay Attack) از ردیس پاک می‌شود.

۶.۲. کش کردن داینامیک نتایج جستجو و لیست بلیط‌ها

برای کاهش بار پردازشی روی هارد دیسک دیتابیس، نتایج جستجوی بلیط‌ها و اطلاعات پروفایل کاربران کش می‌شوند:

- کش پروفایل کاربر: پروفایل کاربران تحت کلید `user:profile:[id]` با زمان اعتبار ۱ ساعت کش می‌شود.
- کش داینامیک جستجو: به دلیل گستردگی پارامترهای فیلتر در جستجوی بلیط (ورزش، تیم، شهر، قیمت و...)، یک الگوریتم تولید هش MD5 بر روی پارامترهای ارسالی کلاینت پیاده‌سازی شده است. خروجی این هش، کلید کش ردیس را به فرمت `tickets:search:[hash]` تعریف می‌کند تا نتایج جستجوهای مشابه تا ۱ دقیقه بدون مراجعه به دیتابیس از رم خوانده شوند.

۶.۳. ابطال هدفمند کش (Cache Invalidation)

بزرگ‌ترین چالش در کار با سیستم‌های کش، همگام نگه داشتن داده‌ها (Data Consistency) است. در این پروژه استراتژی ابطال فعال کش به صورت دقیق پیاده‌سازی شده است:

- به محض آپدیت پروفایل توسط کاربر، کلید کش پروفایل او بلافاصله پاک می‌شود (`redisClient.del`).
- به محض ثبت رزرو موقت یا نهایی شدن پرداخت یک بلیط، کش جزئیات آن بلیط و تمام کلیدهای جستجوی بلیط‌ها (`tickets:search:*`) با استفاده از الگوهای جستجوی کلید در ردیس ابطال می‌شوند تا کلاینت‌ها ظرفیت‌های منقضی‌شده را مشاهده نکنند.

۶.۴. مکانیزم رزرو موقت (Ticket Lock & Expiration Job)

مکانیزم قفل موقت بلیط به منظور جلوگیری از رزرو بی‌پایان و مسدود شدن ظرفیت‌ها طراحی شده است:

- کاربر پس از انتخاب بلیط، آن را با وضعیت `pending` و زمان انقضای ۱۰ دقیقه‌ای رزرو می‌کند.

- یک سرویس پس‌زمینه (Background Job) به صورت خودکار در وب‌سرور فعال است که هر ۱ دقیقه یک‌بار، رزروهای منقضی‌شده را در دیتابیس شناسایی کرده، وضعیت آن‌ها را به expired تغییر داده و ظرفیت صندلی مربوطه را مجدداً به دیتابیس بازمی‌گرداند.

۷. تست‌نویسی واحد و یکپارچه‌سازی لایه سرور (Backend Testing)

تضمین صحت کارکرد کدهای بک‌اند با پیاده‌سازی ۲۲ تست مجزا و خودکار در فریم‌ورک Jest با هماهنگی کامل با معماری ES Modules انجام گرفت.

۷.۱. تست‌های واحد لایه سرویس (Unit Tests)

در بخش تست‌های واحد (پوشه tests/unit/)، اتصالات واقعی به دیتابیس پستگرس و ردیس با استفاده از قابلیت شبیه‌سازی پیشرفته Jest (jest.unstable_mockModule) به طور کامل مسدود شدند.

- تست‌های واحد بلیط: صحت لاجیک‌های خواندن از کش ردیس در صورت وجود، و مراجعه به دیتابیس در صورت Cache Miss، در کنار صحت ابطال کش بررسی شد.
- تست‌های واحد جریمه کنسلی: قوانین جریمه کنسلی بلیط به صورت سناریوهای مرزی زمانی تست شدند (کسر ۱۰٪ جریمه برای فاصله بالای ۲۴ ساعت، ۳۰٪ جریمه برای فاصله بین ۳ تا ۲۴ ساعت، ۱۰۰٪ جریمه برای فاصله کمتر از ۳ ساعت، و رد درخواست برای مسابقاتی که شروع شده‌اند).

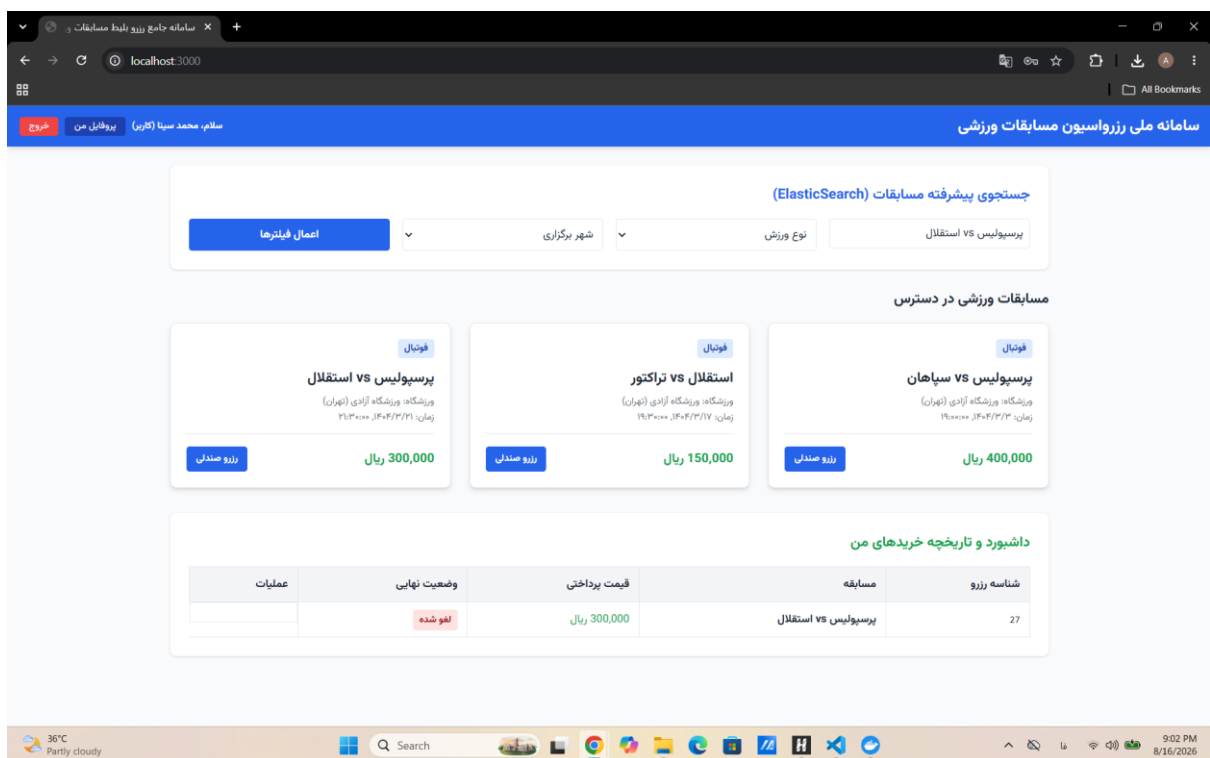
۷.۲. تست‌های یکپارچه‌سازی اندپوینت‌ها (Integration Tests)

در بخش تست‌های یکپارچه‌سازی (پوشه tests/integration/)، کل سیستم وب‌سرور اکسپرس به همراه شبیه‌سازهای درخواست HTTP (supertest) در برابر یک دیتابیس واقعی مورد سنجش قرار گرفت:

- تست‌های گارد امنیتی مسیرها: تایید مسدود شدن دسترسی و بازگشت ارور ۴۰۱ برای درخواست‌های فاقد توکن معتبر JWT در مسیرهای رزرو، پرداخت و ویرایش اطلاعات.
- تست‌های لایه اعتبارسنجی: تایید رد درخواست ثبت‌نام در صورت نامعتبر بودن پسورد (کمتر از ۶ کاراکتر) و بازگشت ارور استاندارد 400 Bad Request مطابق با قوانین اعتبارسنجی کلاس دامنه.
- تست‌های احراز هویت: بررسی سناریوهای تایید کد OTP نامعتبر یا تاریخ گذشته و بازگشت دقیق ارورهای مربوطه.

۸. فاز چهارم: موتور جستجوی الاستیک سرچ و رابط کاربری کلاینت

در این فاز با هدف بهبود کارایی فرآیند جستجو، کاهش فشار روی هارد دیسک پایگاه داده رابطه‌ای (PostgreSQL) در بار ترافیکی بالا، و ارائه یک تجربه کاربری مدرن، موتور جستجوی هوشمند Elasticsearch در کنار یک رابط کاربری وب واکنش‌گرا (Responsive Web UI) پیاده‌سازی گردید.



۸.۱. چالش جستجو در دیتابیس‌های رابطه‌ای (RDBMS)

در سیستم‌های پربازدید، اجرای کوئری‌های متنی سنگین با دستورات LIKE یا ILIKE بر روی جداول حجیم، منجر به قفل شدن سطرها (Row Locking) و افزایش زمان پاسخ‌دهی (Latency) می‌شود. پایگاه داده PostgreSQL برای مدیریت تراکنش‌ها (ACID) بهینه‌سازی شده است، در حالی که Elasticsearch به عنوان یک موتور جستجوی توزیع‌شده با استفاده از ساختار معکوس ایندکس (Inverted Index) برای جستجوهای متنی فوق سریع طراحی گردیده است.

۸.۲. تنظیمات ایندکس‌گذاری و تحلیلگر اختصاصی فارسی (Custom Analyzer)

برای پشتیبانی از جستجوی هوشمند و قابلیت تکمیل خودکار کلمات (Autocomplete) در زبان فارسی، ایندکس پایگاه داده با پی‌کرندهای زیر در لایه Elasticsearch ساخته شد:

- **توکنایزر Edge n-gram:** این توکنایزر با تجزیه کلمات به بخش‌های ۲ تا ۱۵ کاراکتری، امکان جستجوی "تایپ همزمان" (Search-as-you-type) را فراهم می‌آورد.
- **فیلتر نرمال‌سازی فارسی (Persian Normalization):** برای یکپارچه‌سازی حروف خاص فارسی (مانند «ی» و «ک» که در کیبورد‌های مختلف کدهای متفاوتی دارند)، فیلترهای استاندارد به مپینگ فیلدها متصل شدند.
- **مپینگ ویژگی‌های متنی:** فیلدهای نام تیم میزبان، تیم میهمان و نام ورزشگاه مپینگ دوگانه‌ای دریافت کردند که شامل فیلد اصلی برای تحلیل دقیق و یک زیرفیلد اختصاصی autocomplete. مجهز به این توکنایزر برای پیشنهادات زنده است.

۸.۳. استراتژی دوگانه همگام‌سازی داده‌ها (Data Synchronization)

برای تضمین یکپارچگی اطلاعات و جلوگیری از هرگونه مغایرت ظرفیت صندلی‌ها بین PostgreSQL و Elasticsearch، از دو سناریوی همگام‌سازی استفاده شده است:

۱. **همگام‌سازی سرد در زمان استارت‌آپ (Cold Sync):** به محض بالا آمدن وب‌سرور، یک جاب سراسری اجرا شده و تمامی بلیط‌های موجود در دیتابیس SQL را به طور کامل در الاستیک‌سرچ بازنویسی (Reindex) می‌کند تا از همگام بودن اولیه ساختار مطمئن شویم.

۲. **همگام‌سازی گرم لحظه‌ای (Real-time Sync):** به محض ثبت رزرو موقت، پرداخت نهایی بلیط، یا لغو تراکنش توسط کاربر، سرویس بک‌اند بلافاصله متد syncTicket را صدا می‌زند تا ظرفیت جدید صندلی و وضعیت بلیط را در کسری از ثانیه در سند متناظر الاستیک‌سرچ بروزرسانی کند.

۸.۴. پیاده‌سازی پیشنهادات هوشمند (Autocomplete Suggester)

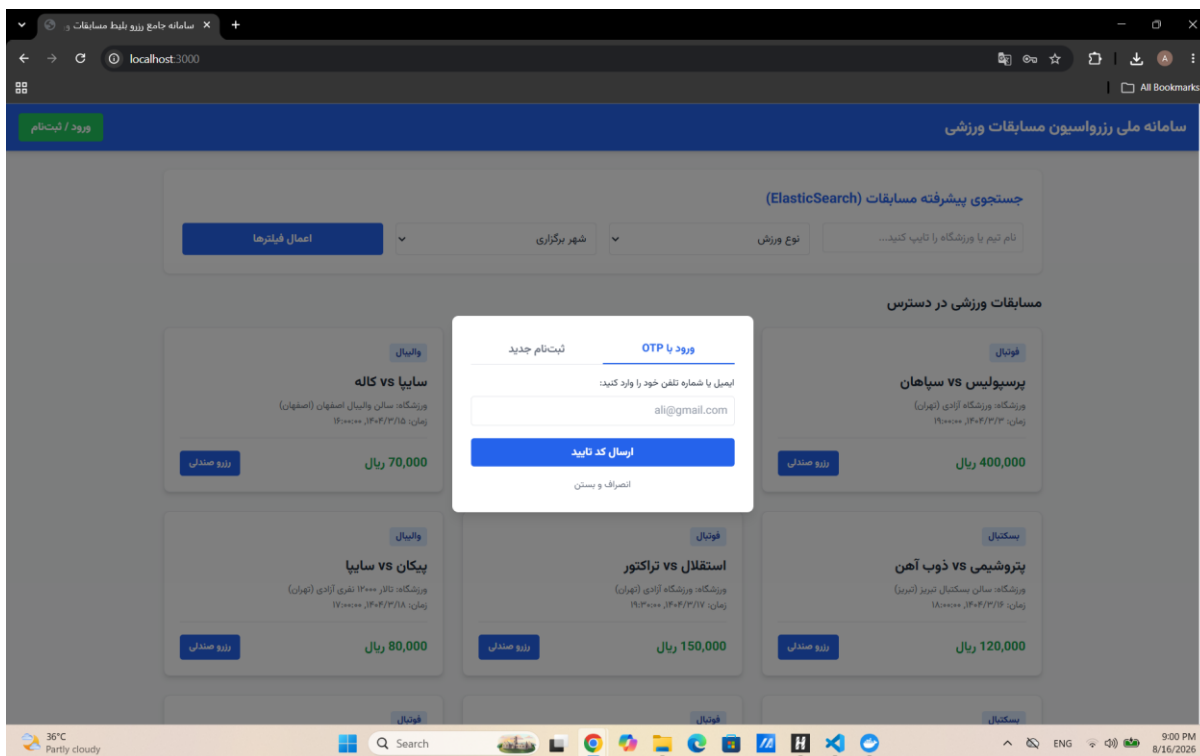
اندپوینت اختصاصی `/api/tickets/autocomplete` با استفاده از کوئری هوشمند `multi_match` از نوع `bool_prefix` در الاستیک‌سرچ توسعه داده شد. این اندپوینت به کلاینت اجازه می‌دهد به محض تایپ حداقل ۲ کاراکتر توسط کاربر، سریعاً پیشنهادات مرتبط با مسابقات فعال را بدون ارسال کوئری به دیتابیس اصلی، از الاستیک استخراج کند.

۹. معماری رابط کاربری کلاینت (SPA Web Frontend)

رابط کاربری کلاینت به صورت یک نرم‌افزار تک‌صفحه‌ای (Single Page Application) سبک و پویا با بهره‌گیری از **Tailwind CSS** و کدهای جاوااسکریپت خالص (Vanilla JS) پیاده‌سازی شد. مزیت این رویکرد، عدم وابستگی به ابزارهای پیچیده بیلد (همچون وب‌پک یا ویت) و سرعت لود فوق‌العاده بالا به دلیل سرو شدن مستقیم فایل‌ها از لایه استاتیک وب‌سرور اکسپرس است.

۹.۱. بخش‌های کلیدی رابط کاربری

- **فرم ورود هوشمند با OTP:** شبیه‌سازی کامل درخواست کد تایید، ذخیره‌سازی توکن امنیتی JWT در `LocalStorage` مرورگر و بازیابی پویا و حفظ نشست کاربر در مراجعات بعدی.



- **موتور جستجوی داینامیک:** اتصال لایه ورودی به متد Autocomplete الاستیکسرچ به صورت رویداد زنده oninput و رندر کردن زنده باکس پیشنهادات.
- **داشبورد مدیریت رزروها:** نمایش جزئیات بلیط‌های خریداری شده، رزروهای معلق در انتظار پرداخت و رزروهای منقضی یا کنسل شده.
- **فرآیند تعاملی کنسلی و محاسبه جریمه:** کلاینت پیش از تایید نهایی لغو بلیط، جریمه کنسلی را از لایه سرور بر اساس زمان باقی‌مانده تا شروع مسابقه استعلام کرده و پس از تایید کاربر، فرآیند لغو تراکنش و استرداد وجه پس از کسر جریمه را نهایی می‌کند.

۱۰. تضمین کیفیت و تست‌های اختصاصی الاستیکسرچ

تضمین صحت کارکرد لایه جستجو و پیشنهادات هوشمند الاستیکسرچ با پیاده‌سازی تست‌های جدید و افزایش تعداد کل سناریوهای تست بک‌اند به ۲۸ تست موفق انجام گرفت.

۱۰.۱. تست‌های واحد لایه الاستیکسرچ (tests/unit/elasticsearchService.test.js)

در این بخش، برای پایداری فرآیند تست واحد، اتصالات واقعی شبکه به پورت ۹۲۰۰ الاستیکسرچ با استفاده از قابلیت ESM Mocking در جِست مسدود شد:

- **تست ساخت ایندکس:** صحت بررسی وجود ایندکس و اعمال مپینگ فارسی در زمان راه‌اندازی بررسی شد.
- **تست موتور جستجو:** تایید فرمت بدنه کوئری‌های فازی، فیلترها و مرتب‌سازی زمان مسابقات.
- **تست تکمیل خودکار:** تایید ساختار کوئری‌های bool_prefix برای اندپوینت Autocomplete.

۱۰.۲. تست‌های یکپارچه‌سازی وب API

اندپوینت پیشنهادات به همراه تایید مسدود بودن مسیرهای ادمین و کلاینت در برابر توکن‌های نامعتبر، با استفاده از ابزار **Supertest** بررسی شدند. با اضافه شدن دستور بستن صریح کلاینت الاستیکسرچ (`esClient.close()`) در لایه‌ی Teardown تست‌ها، تمام کانکشن‌های شبکه پس از پایان آزمون‌ها با موفقیت بسته شده و پایداری کامل فرآیند تست بدون هیچ‌گونه هشدار یا اتلاف پروسه تضمین گردید.